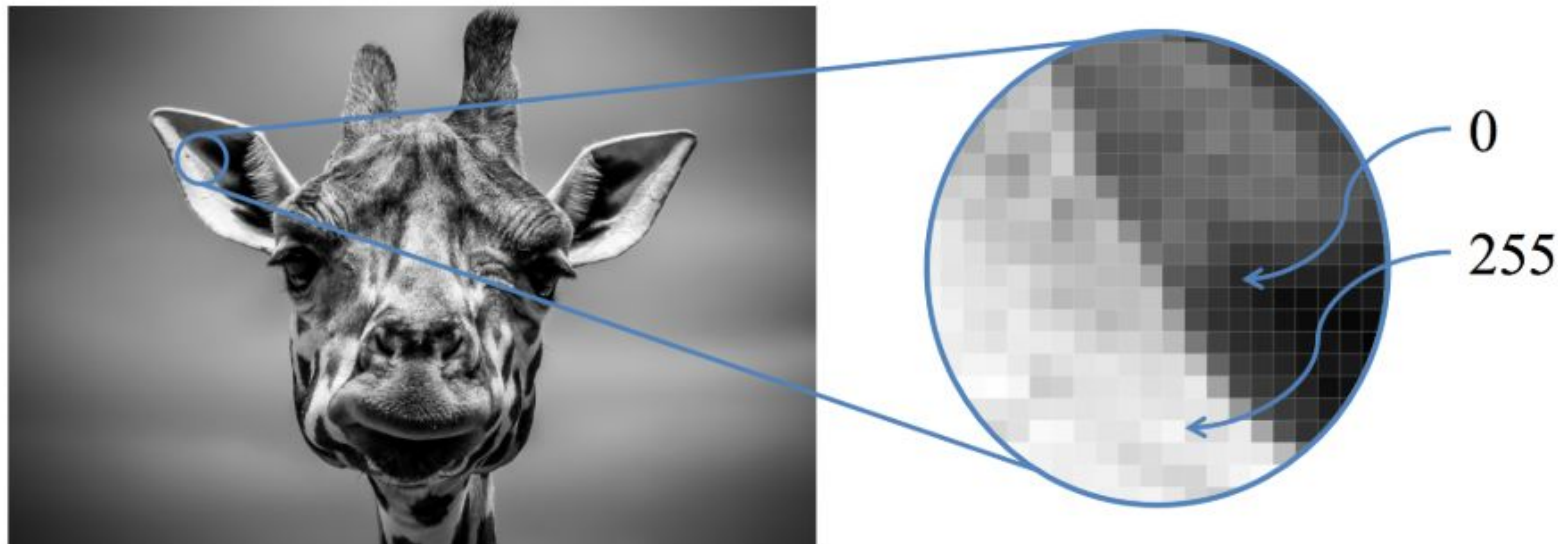


Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

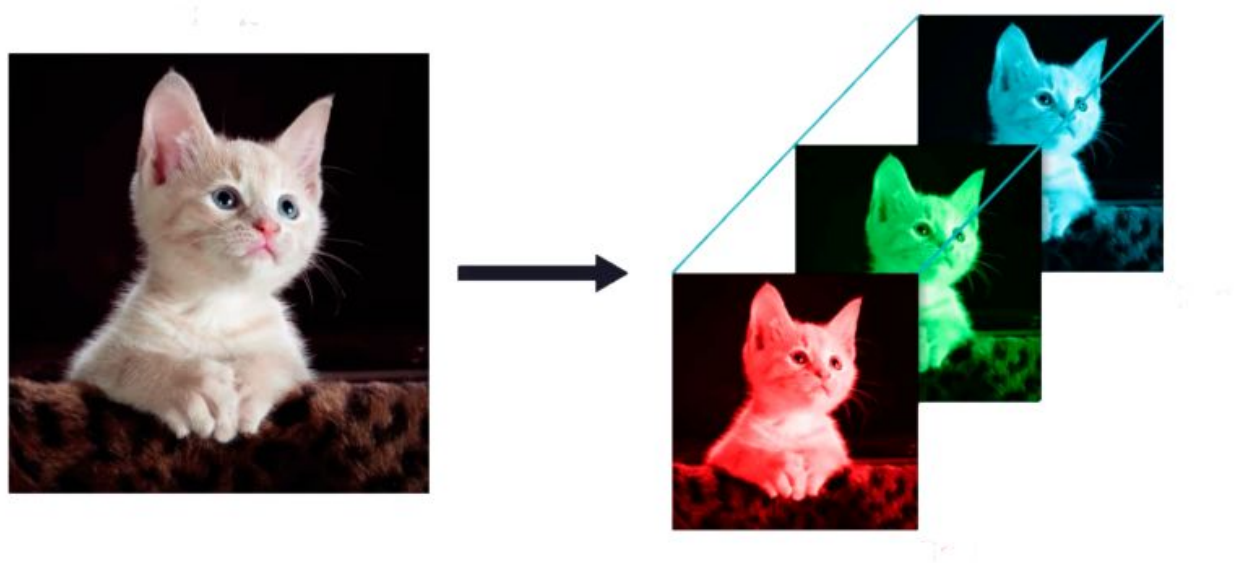
Картинка — тензор

- Каждая картинка — это матрица из пикселей
- Каждый пиксель обладает яркостью по шкале от 0 до 255

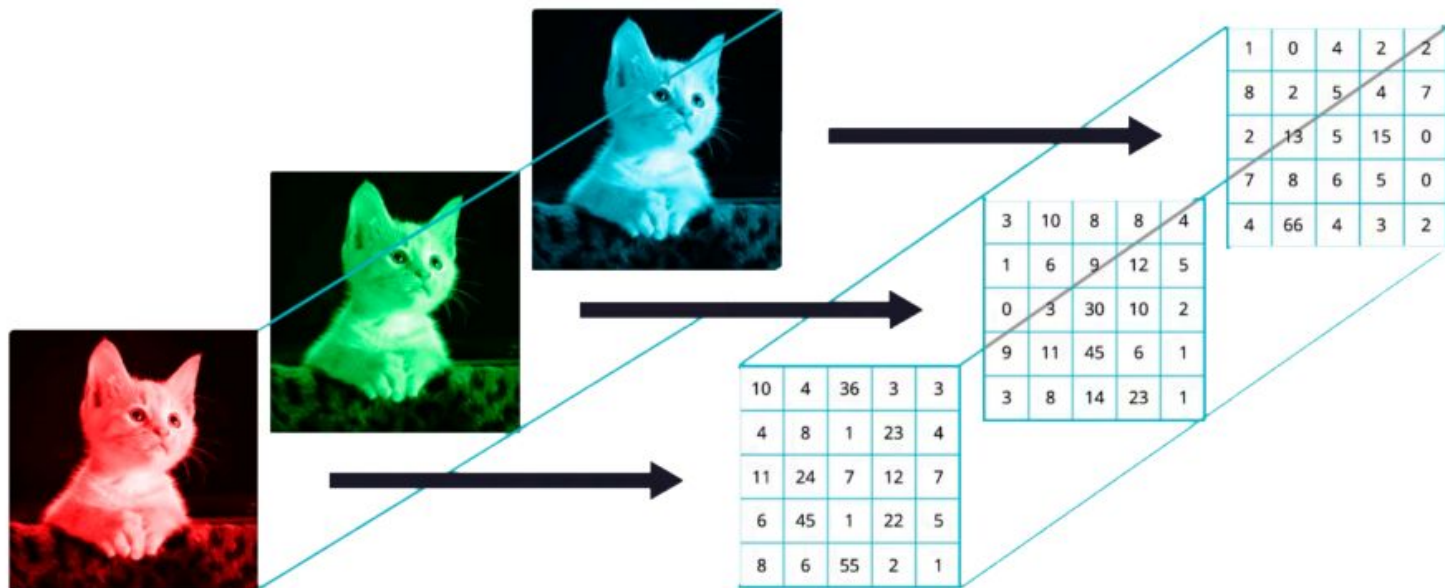


Картинка — тензор

- Цветное изображение имеет три канала яркости: красный, зелёный и синий (rgb)



Картинка — тензор

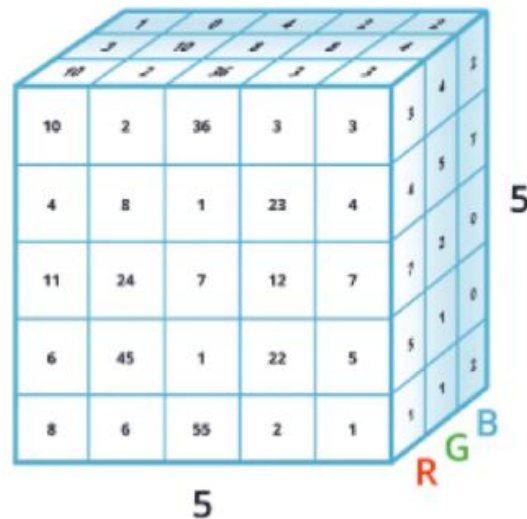


Картинка — тензор

5 x 5 x 3



3D Array



Cifar100

- Набор данных, состоящий из цветных изображений 100 классов
- Размер 32 на 32 пикселя
- 3 цвета

airplane



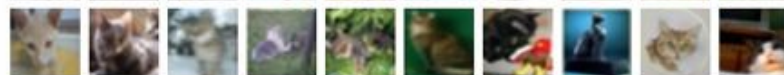
automobile



bird



cat



deer



dog



frog



horse



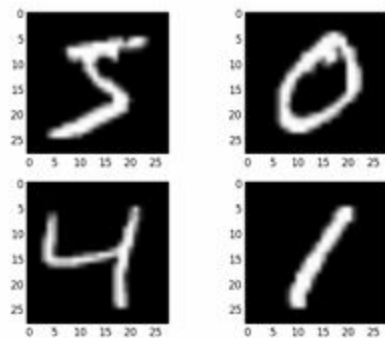
ship



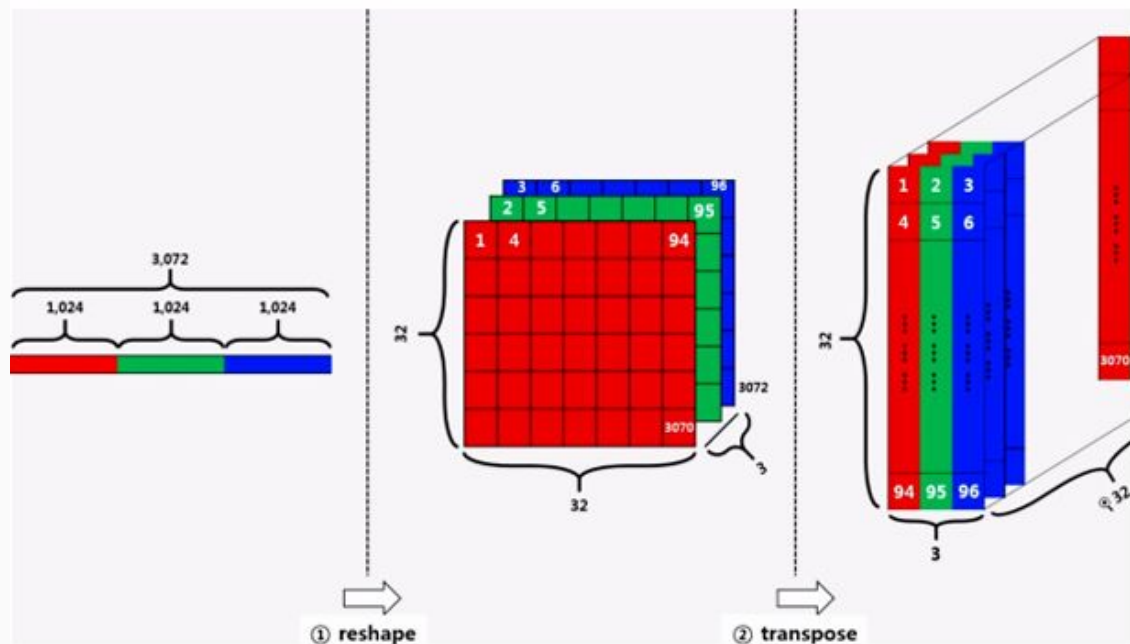
truck



MNIST

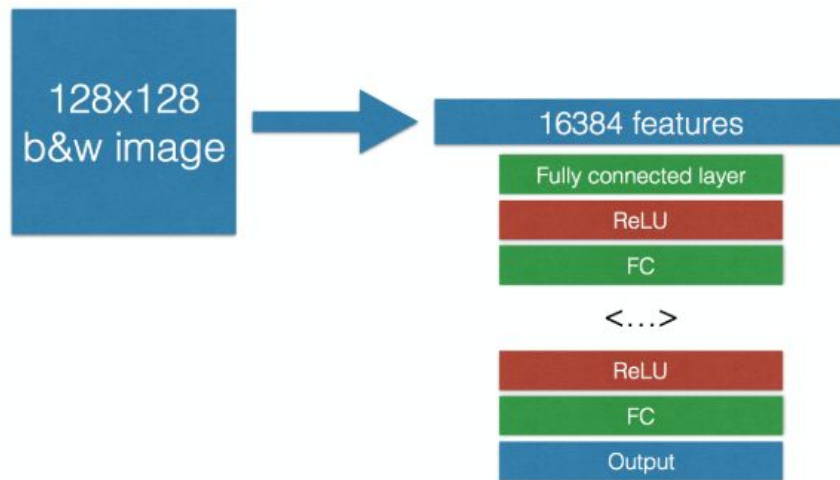


*Reshape, transpose



- Reshape – изменение размерности матрицы
- Transpose – транспонирование
- Количество элементов в матрице остается прежним - 3072

Обычная сетка



- Развернём картинку в вектор $\Rightarrow$ **очень много весов**
- Если изображение немного сдвинуть, то нейрон уже не будет на него реагировать

Полносвязные слои для изображений

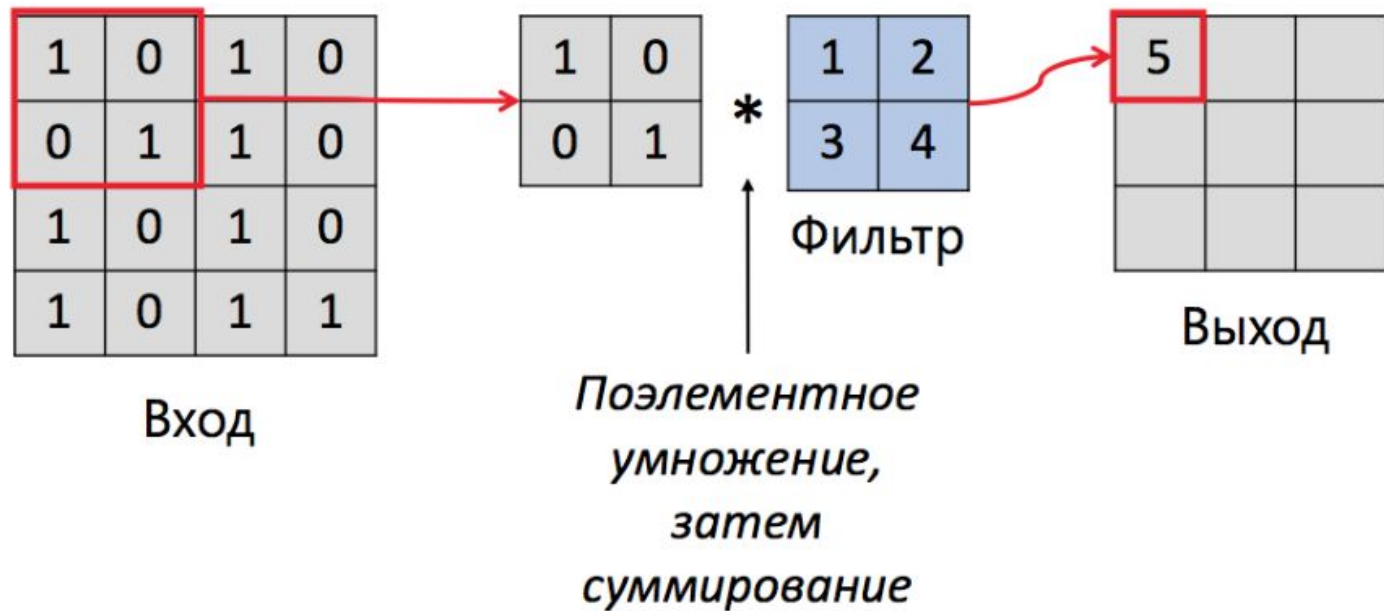
- Очень много параметров
- Легко могут переобучиться
- Не учитывают специфику изображений (сдвиги, небольшие изменения формы и т.д.)
- Изображение в разных местах картинки даёт разные веса
- Теряется информация о взаимном расположении пикселей

Мы хотим, чтобы...

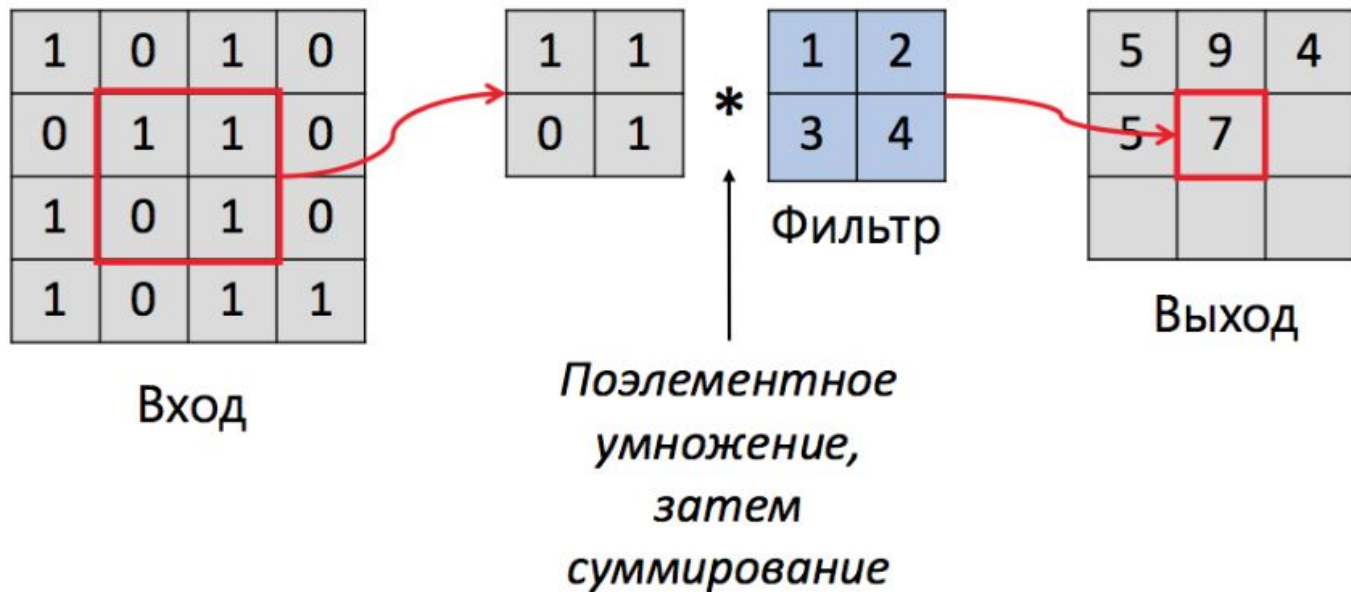
- Хотим меньше параметров, избежать переобучения
- Хотим, чтобы информация не терялась
- Хотим, чтобы модель была нечувствительна к сдвигам картинки в новые места

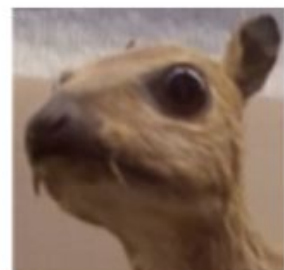
⇒ свёртка

Свёртка



Свёртка



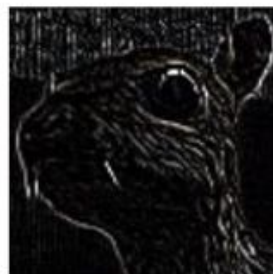


Входная
картинка

Фильтр

$$\begin{matrix} * & \begin{matrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{matrix} \end{matrix}$$

=



Поиск краев

$$\begin{matrix} * & \begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix} \end{matrix}$$

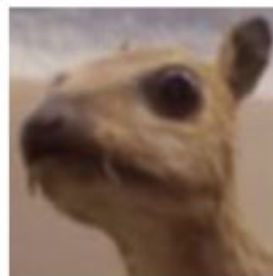
=



Повышение
резкости

$$\begin{matrix} * & \frac{1}{9} & \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix} \end{matrix}$$

=

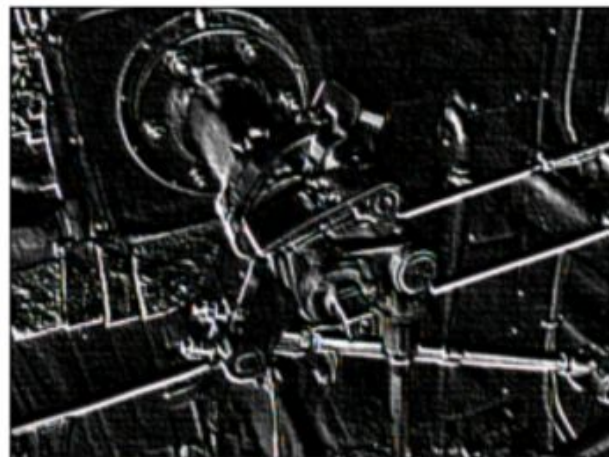
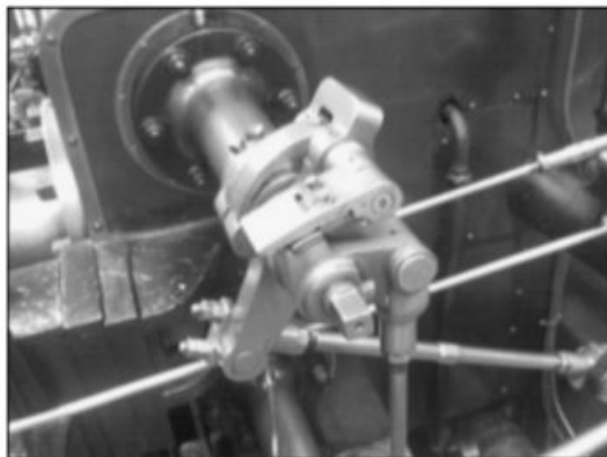


Размытие

Выделение границ

$$\begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

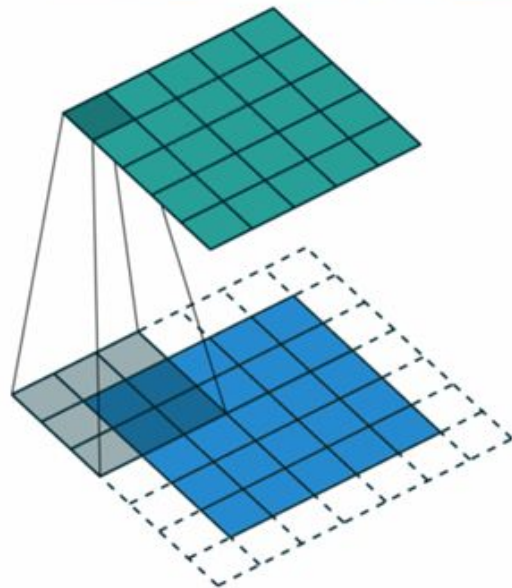


Свёртка

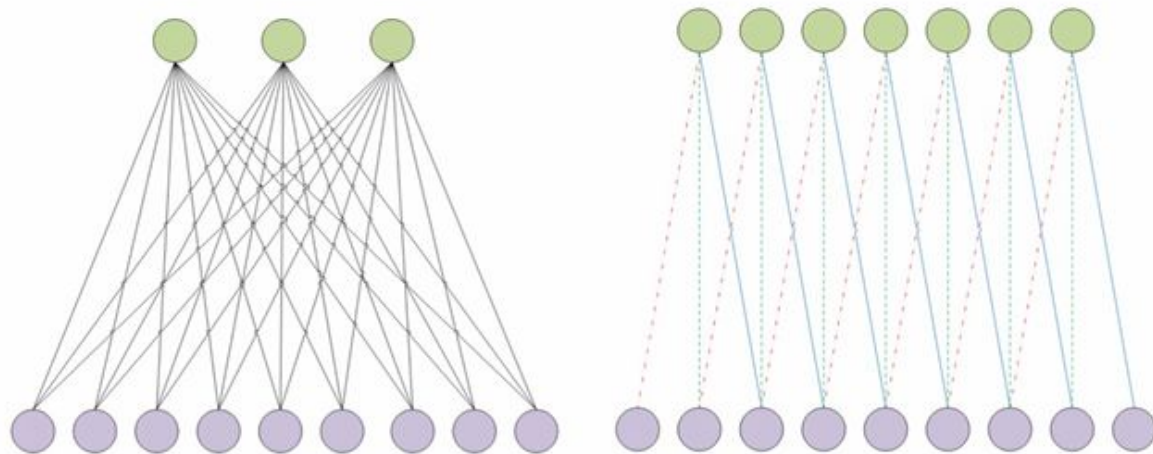
- Операция свёртки выявляет наличие на изображении паттерна, который задаётся фильтром
- Результат операции свёртки — новое изображение
- Чем сильнее на участке изображения представлен паттерн, тем больше будет значение свёртки
- **Идея:** дать нейросети возможность самостоятельно придумать ядро для поиска нужных ей паттернов

Свертка

- Свертка – это применение одного и того же фильтра (нейрона) к разным частям исходного изображения
- В результате на разных частях исходных данных идет поиск одинаковых признаков
- Выходными данными свертки являются карты признаков
- Несколько фильтров позволяют сформировать несколько карт признаков одного сверточного слоя



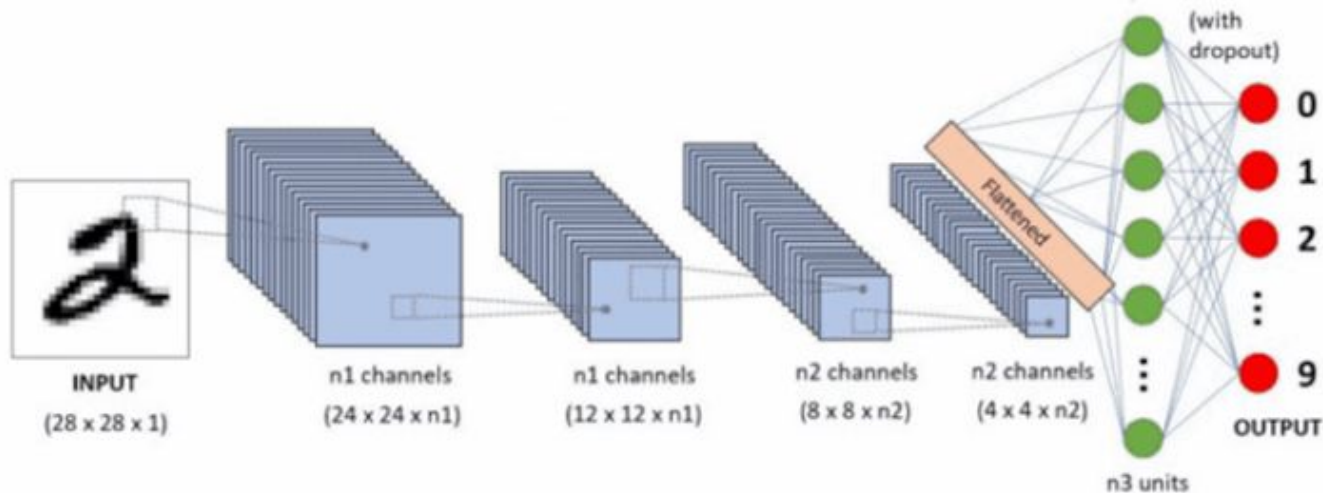
*Свойства сверточного слоя



- Разреженные взаимодействия – каждый нейрон связан с ограниченным числом входных нейронов
- Разделение параметров – в карте признаков все нейроны имеют одинаковый набор параметров
- Инвариантность – сдвиг исходных данных вызывает аналогичный сдвиг значений выходного слоя

*Сверточная нейросеть

- Сверточная нейросеть состоит из нескольких слоев: свертки, пуллинга, полносвязного
- Слои свертки и пуллинга чередуются друг за другом
- Слои свертки применяют набор n_1 , n_2 фильтров к исходному изображению. Каждый фильтр ищет определенные признаки в исходных данных и формируется карта признаков
- Слои свертки обучаются, меняют количество каналов. Вход для свертки трехмерный: $\text{ядро} \times \text{ядро} \times \text{каналы}$
- Слои пуллинга только уменьшают размерность карты признаков, количество каналов сохраняется.
- Данные последнего слоя пуллинга преобразуются в вектор для использования в полносвязном слое



*Вычисления в сверточном слое

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+

+

+ 1 = -25

↑
Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

- Трехмерный случай для трех цветов
- Аналогично несколько каналов слоя свертки
- На входе нейрона 27 значений

Размерность изображения падает

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3 ₀	2 ₁	1 ₂	0
0	0 ₂	1 ₂	3 ₀	1
3	1 ₀	2 ₁	2 ₂	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2 ₀	1 ₁	0 ₂
0	0	1 ₂	3 ₂	1 ₀
3	1	2 ₀	2 ₁	3 ₂
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1 ₀	3 ₁	1 ₂
3	1	2 ₂	2 ₂	3 ₀
2	0	0 ₀	2 ₁	2 ₂
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3 ₀	1 ₁	2 ₂	2	3
2 ₂	0 ₂	0 ₀	2	2
2 ₀	0 ₁	0 ₂	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3	1 ₀	2 ₁	2 ₂	3
2	0 ₂	0 ₂	2 ₀	2
2	0 ₀	0 ₁	0 ₂	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3	1	2 ₀	2 ₁	3 ₂
2	0	0 ₂	2 ₂	2 ₀
2	0	0 ₀	0 ₁	1 ₂

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Дополнение изображения (zero padding)

0_2	0_0	0_1	0	0	0	0
0_1	2_0	2_0	3	3	3	0
0_0	0_1	1_1	3	0	3	0
0	2	3	0	1	3	0
0	3	3	2	1	2	0
0	3	3	0	2	3	0
0	0	0	0	0	0	0

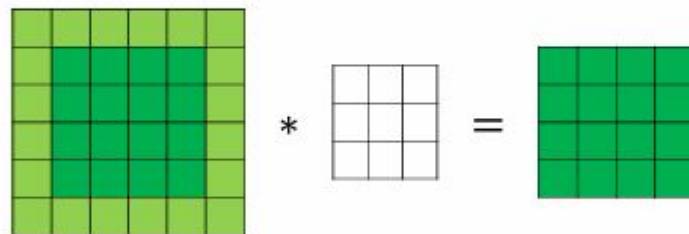
Дополнение (Padding)

- Если применять свёртку по формуле, итоговое изображение будет меньше исходного
- Из-за этого мы можем терять информацию на краях изображения
- Нужно заполнить края рамочкой: zero padding, replication padding, reflection padding
- Есть риск, что модель научится понимать, где на изображении края — можем потерять инвариантность
- Разные типы паддингов допускают разные способы переобучения под края

Свёртки

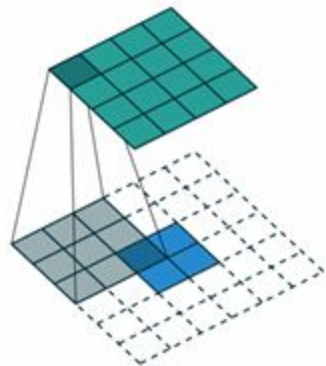
- Если применять свёртку по формуле, то выходное изображение будет меньше входного

Свёртки



*Padding

- Padding – заполнение исходных данных для свертки
- Либо нулями, либо повторение соседних ячеек
- Padding – возможность сохранить размерность карты признаков или даже ее увеличить
- Для сохранения размера, padding равен $(\text{размеру ядра свертки}-1)/2$



0	0	0	0	0	0	0	0
0	2	2	3	3	3	0	0
0	0	1	3	0	3	0	0
0	2	3	0	1	3	0	0
0	3	3	2	1	2	0	0
0	3	3	0	2	3	0	0
0	0	0	0	0	0	0	0

1	6	5
7	10	9
7	10	8

Valid mode

- При честном подсчёте свёрток пиксели на краях не оказывают большого влияния на результат

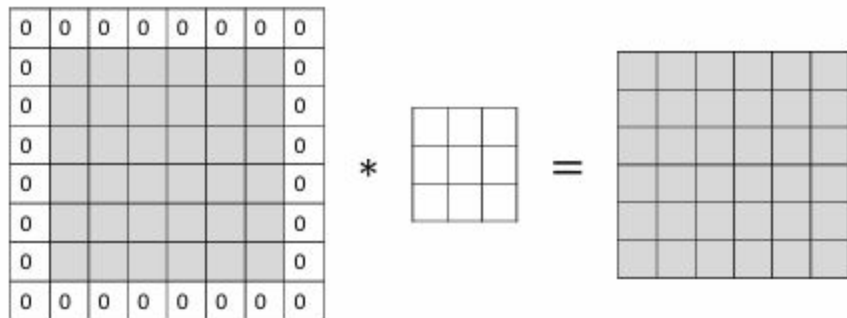
Не увидим, что фильтр имеет хороший отклик при помещении центра в этот пиксель

0	1				
1	1				

*

0	0	1
0	0	1
1	1	1

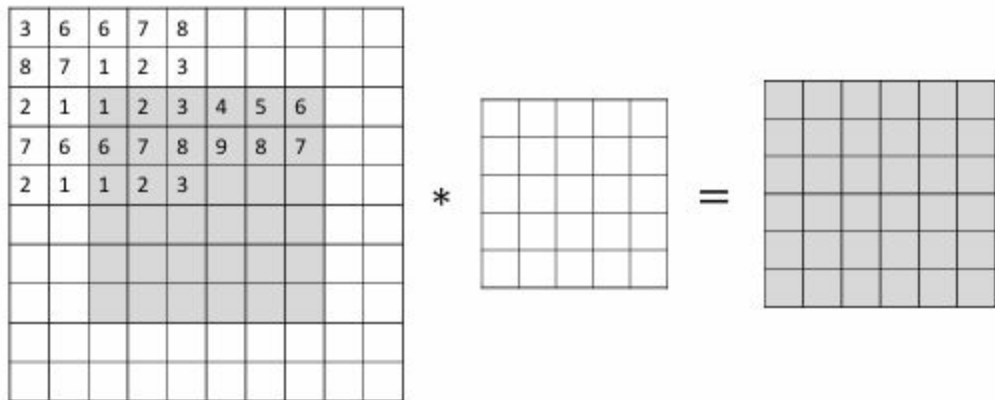
Zero padding



Zero padding

- Добавляем по границам нули так, чтобы посчитанная после этого свёртка в valid mode давала изображение такого же размера, как исходное
- Есть риск, что модель научится понимать, где на изображении края — можем потерять инвариантность

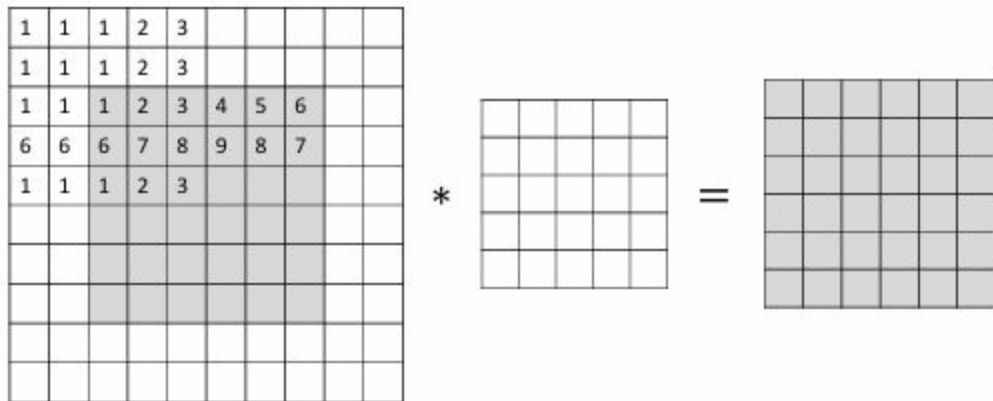
Reflection padding



Reflection padding

- Не получится легко находить края изображения
- Но теперь модель может начать находить зеркальные отражения и подбирать фильтры под них

Replication padding



Резюме

- Паддинг позволяет контролировать размер выходных изображений
- Паддинг позволяет учитывать даже объекты на краях
- Разные типа паддингов допускают разные способы переобучения под края

Replication padding

- Пиксель на границе равен ближайшему пикселю из изображения
- Модель всё ещё может настроиться под паттерны, которые возникают из-за такого паддинга

Свёртка в виде формулы

Чёрно-белый случай:

$$out[x, y] = \sum_{i=-d}^d \sum_{j=-d}^d K[i, j] \cdot Image[x + i, y + j]$$

Цветной случай:

$$out[x, y] = \sum_{i=-d}^d \sum_{j=-d}^d \sum_{c=1}^C K[i, j, c] \cdot Image[x + i, y + j, c]$$

В черно-белом случае свертка вычисляется как сумма произведений элементов ядра свертки $K[i, j]$ на соответствующие пиксели входного изображения $Image[x + i, y + j]$.

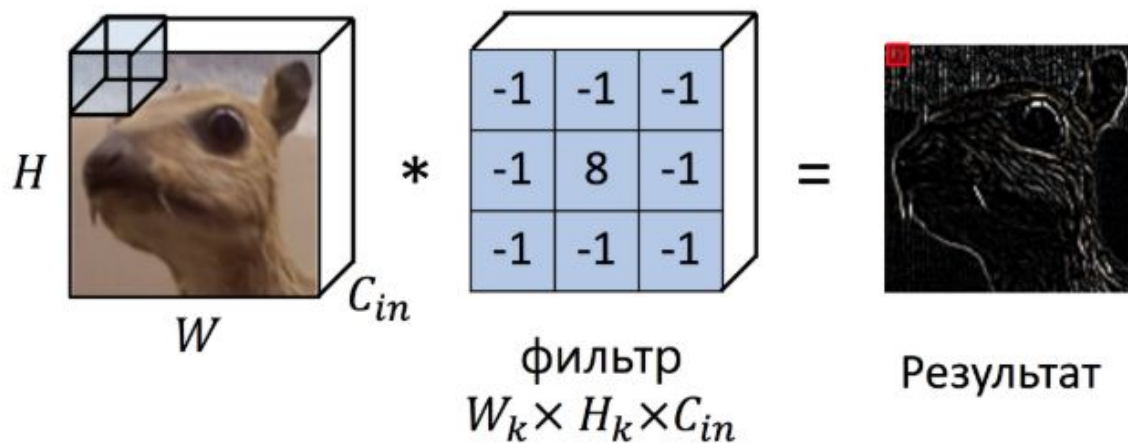
В цветном случае свертка вычисляется как сумма произведений элементов ядра свертки $K[i, j, c]$ на соответствующие пиксели входного изображения $Image[x + i, y + j, c]$ для каждого цветового канала c .

В приведенных формулах для свертки в черно-белом и цветном случае используются следующие переменные:

- out - выходное значение свертки для определенного пикселя $[x, y]$ на выходном изображении.
- x, y - координаты пикселя на выходном изображении.
- d - размер ядра свертки (kernel), которое используется для вычисления свертки. Ядро свертки - это матрица, содержащая веса, которые используются для вычисления линейной комбинации пикселей вокруг текущего пикселя на входном изображении.
- i, j - индексы пикселей вокруг текущего пикселя на входном изображении, которые используются для вычисления свертки. Индексы i и j изменяются от $-d$ до d , что означает, что ядро свертки перемещается по всему входному изображению, вычисляя линейную комбинацию пикселей вокруг текущего пикселя.
- K - ядро свертки (kernel), которое используется для вычисления свертки. В черно-белом случае ядро свертки является двумерной матрицей, а в цветном случае ядро свертки является трехмерной матрицей.
- $Image$ - входное изображение, которое подвергается свертке. В черно-белом случае изображение является двумерной матрицей, а в цветном случае изображение является трехмерной матрицей.
- c - индекс цветового канала в цветном случае. В цветном случае ядро свертки и входное изображение имеют три цветовых канала (например, RGB), и свертка вычисляется для каждого цветового канала отдельно.

Свёртка для цветной картинки

- Цветная картинка — это тензор $W \times H \times C_{in}$
- W — высота, H — ширина, C_{in} — число каналов



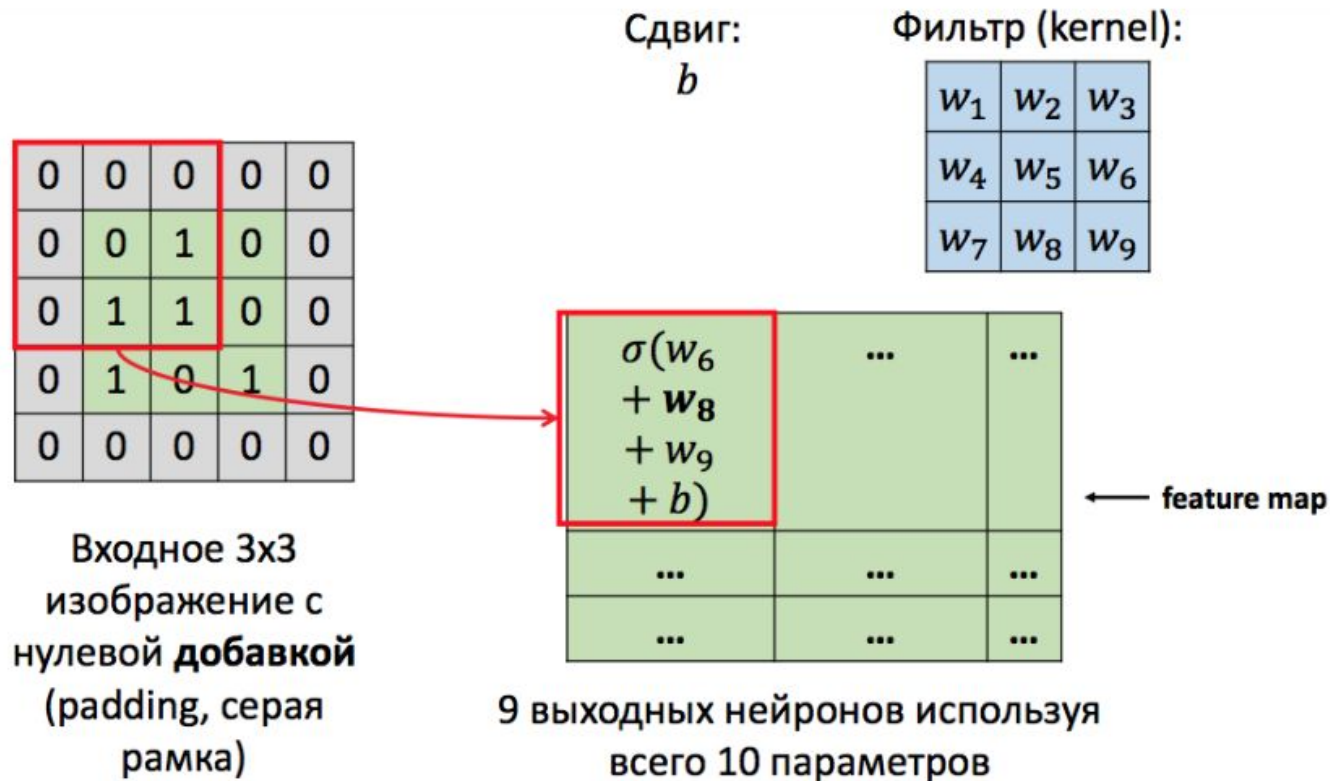
Резюме по свёрткам

- Операция свёртки выявляет наличие на изображении паттерна, который задаётся конкретным фильтром (ядром)
- Чем сильнее на участке изображения представлен паттерн, тем больше будет значение свёртки
- Результат свёртки изображения с фильтром — новое изображение
- Нас будет интересовать много различных паттернов $\Rightarrow$ будем использовать несколько свёрток сразу

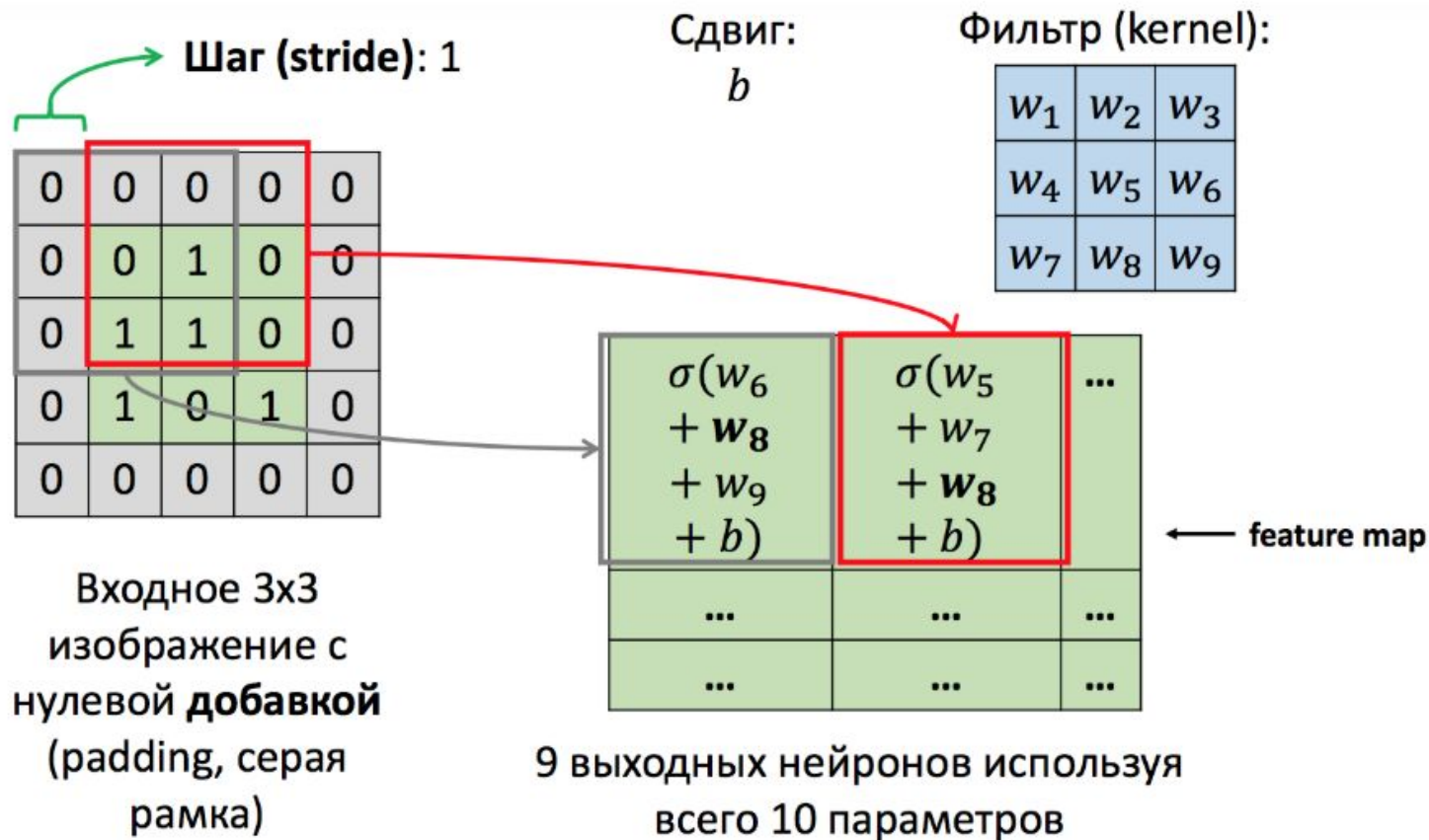
Свёрточный слой

Идея: дать нейросети возможность самостоятельно придумать ядро для поиска нужных ей паттернов

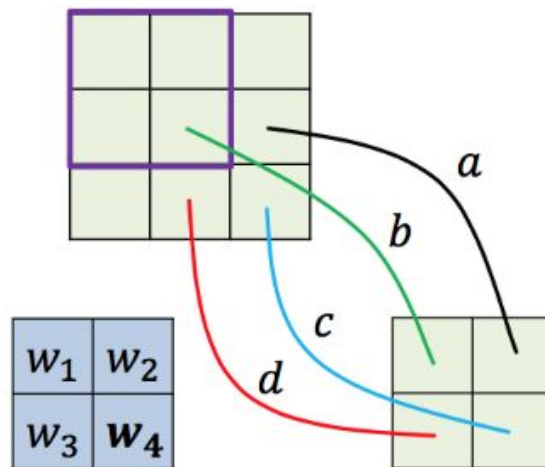
Свёрточный слой



Свёрточный слой



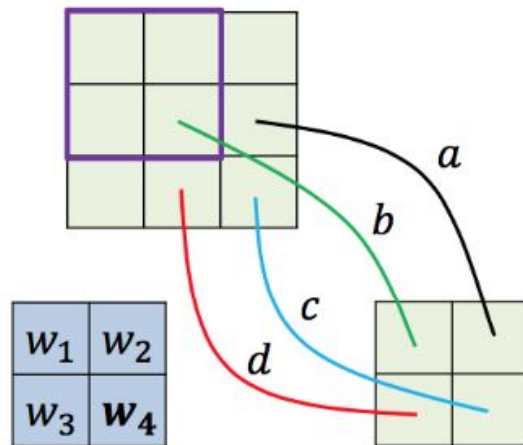
Как считать градиенты



$$b = w_1x_{11} + w_2x_{12} + w_3x_{21} + w_4x_{22}$$

$$a = w_1x_{12} + w_2x_{13} + w_3x_{22} + w_4x_{23}$$

Как считать градиенты



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \cdot x_{11} + \frac{\partial L}{\partial b} \cdot x_{12} + \frac{\partial L}{\partial c} \cdot x_{21} + \frac{\partial L}{\partial d} \cdot x_{22}$$

Суммируем градиенты по всем использованиям веса w_4

Свёрточный слой

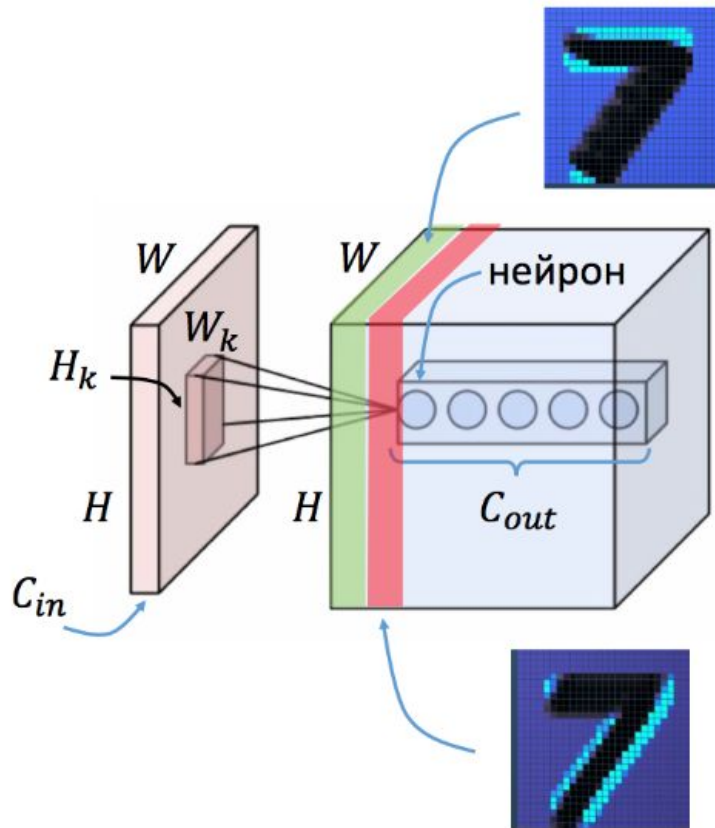
- Слой действует одинаково для каждого участка картинки, в отличие от полносвязного
- Нужно оценивать меньшее количество параметров
- Слой учитывает взаимное расположение пикселей
- Можно учить тем же самым алгоритмом обратного распространения ошибки
- Свёрточный слой — это полносвязный слой с ограничениями

В свёрточном слое меньше параметров

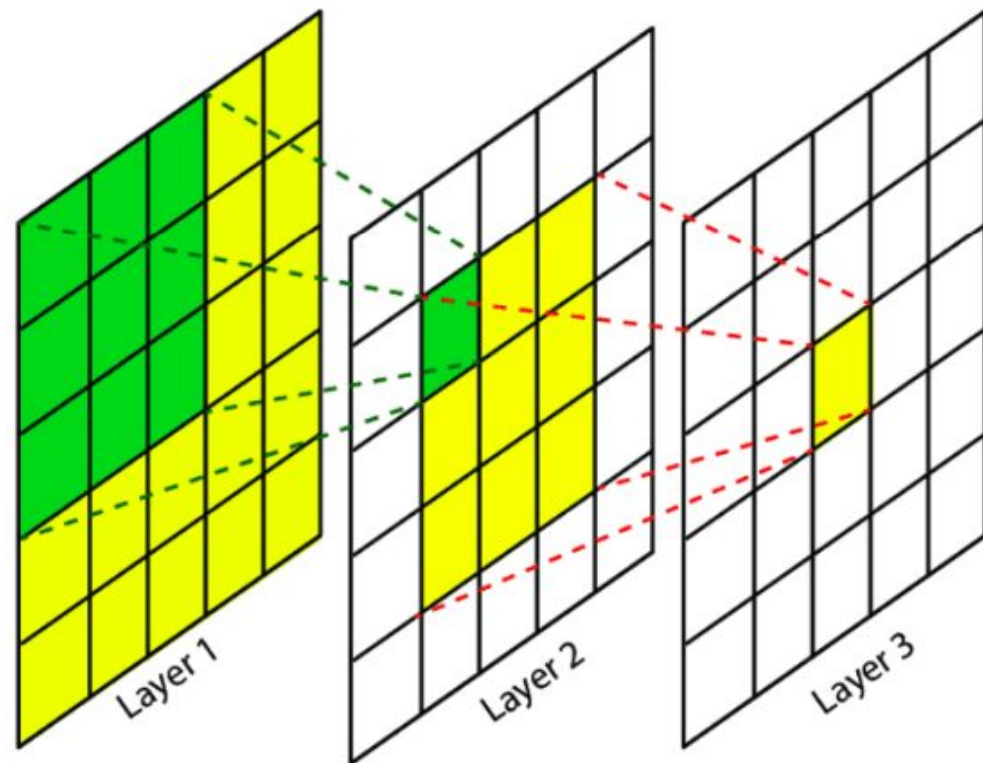
- Картинка 300×300
- 300×300 нейронов
- Свертка 5×5 – 26 параметров
- В полносвязном слое – $8.1 \cdot 10^9$ параметров

Одного фильтра мало!

- На входе $W \times H \times C_{in}$
- На выходе $W \times H \times C_{out}$
- В одном пикселе появляется глубина с разными характеристиками пикселя изображения
- Сколько параметров надо оценить?

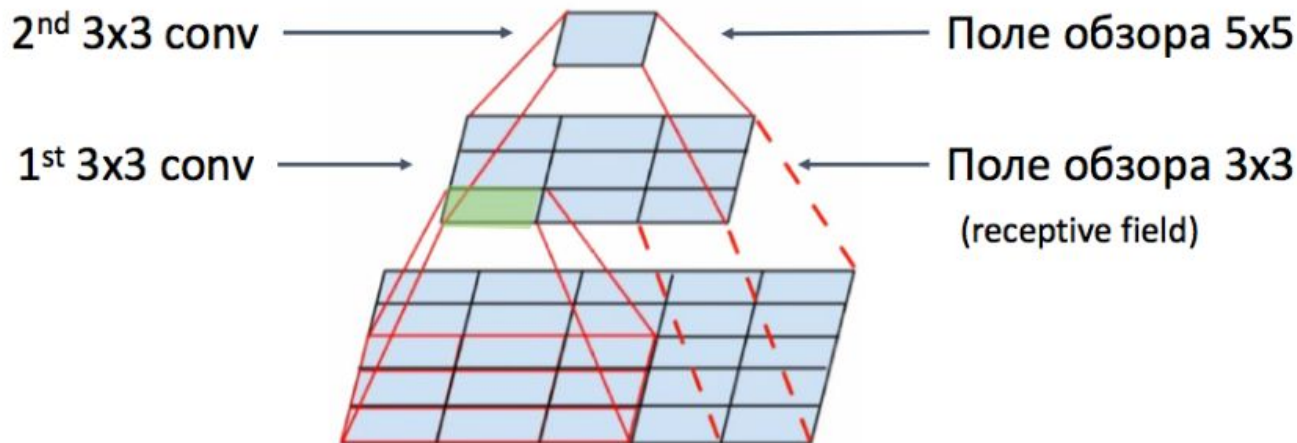


Поле видимости (receptive field)



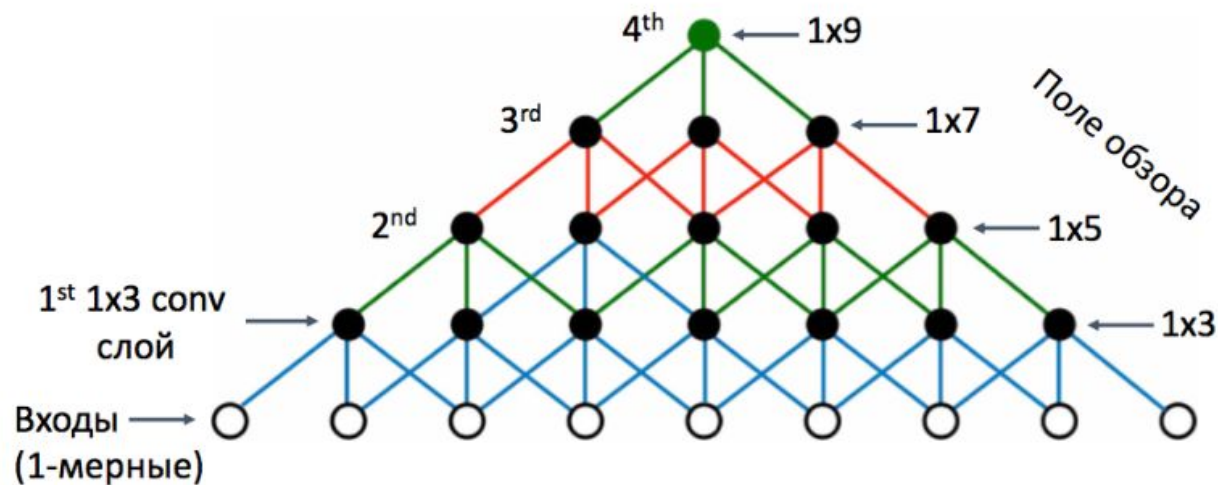
Одного свёрточного слоя недостаточно

- Нейроны первого слоя смотрят на поле 3×3
- Если интересующий нас объект больше, нам нужна вторая свёртка



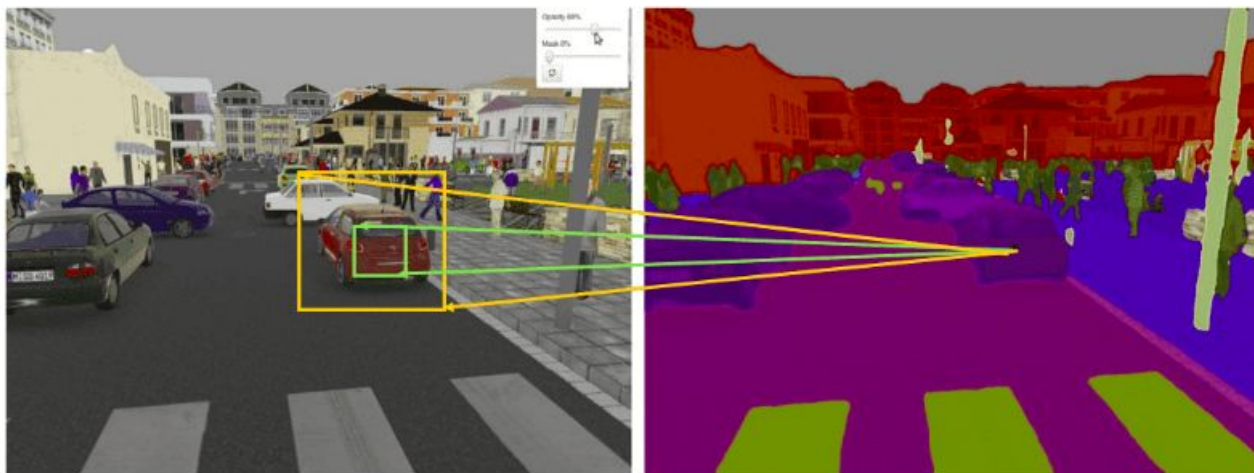
Одного свёрточного слоя недостаточно

- N слоёв со свёртками 3×3
- На N -ом слое поле обзора $(2N + 1) \times (2N + 1)$
- Если наш объект размера 300×300 , надо 150 слоёв...



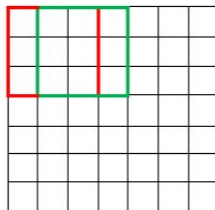
Помоги CNN найти машину

- Наша цель - детекция автомобилей
- Оранжевым и зелёным выделены два разных поля видимости
- Сеть с маленьким полем видимости не сможет распознать машину

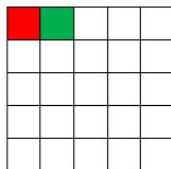


Свёртки с пропусками (strides)

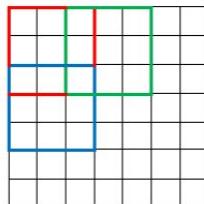
7 x 7 Input Volume



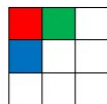
5 x 5 Output Volume



7 x 7 Input Volume



3 x 3 Output Volume

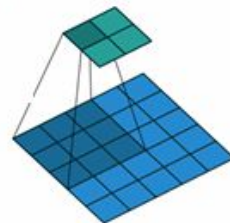


* Шаг свертки - stride

- Stride - шаг свертки
- Это регулируемый параметр, который определяет размерность карты признаков

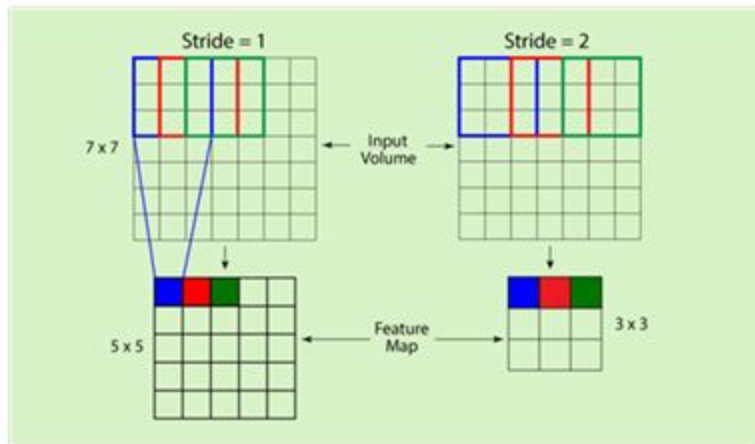
0 ₂	0 ₀	0 ₁	0	0	0	0
0 ₁	2 ₀	2 ₀	3	3	3	0
0 ₀	0 ₁	1 ₁	3	0	3	0
0	2	3	0	1	3	0
0	3	3	2	1	2	0
0	3	3	0	2	3	0
0	0	0	0	0	0	0

1	6	5
7	10	9
7	10	8



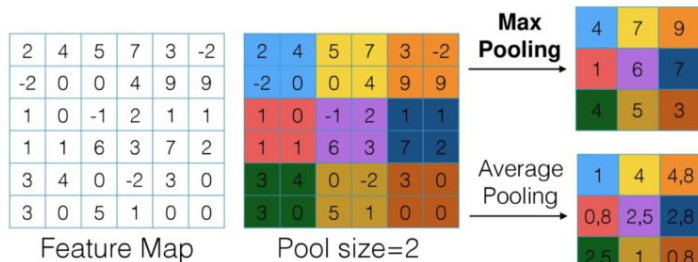
*Stride

- Stride - это еще один способ сократить размерность карты признаков
- Чем больше шаг, тем меньше становится итоговая карта признаков



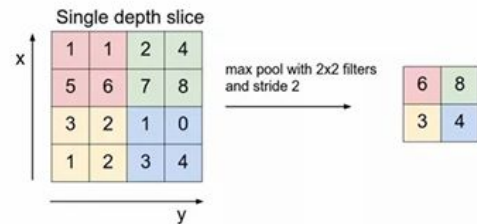
Пулинг слой (Pooling)

- Внутри какого-то окна ищем максимум или среднее, размер изображения существенно сокращается, а поле восприятия растёт



*Пулинг

- Слой пуллинга позволяет сократить размерность карты признаков
- Первый вариант пуллинга - максимальное значение из нескольких соседних



Pooling

1	0	2	1	0	0
0	1	3	2	1	2



1	3	2

Max-pooling с фильтром 2x2

*Average Pooling

- Второй вариант вычисления – среднее значение из нескольких соседних

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool
→
Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

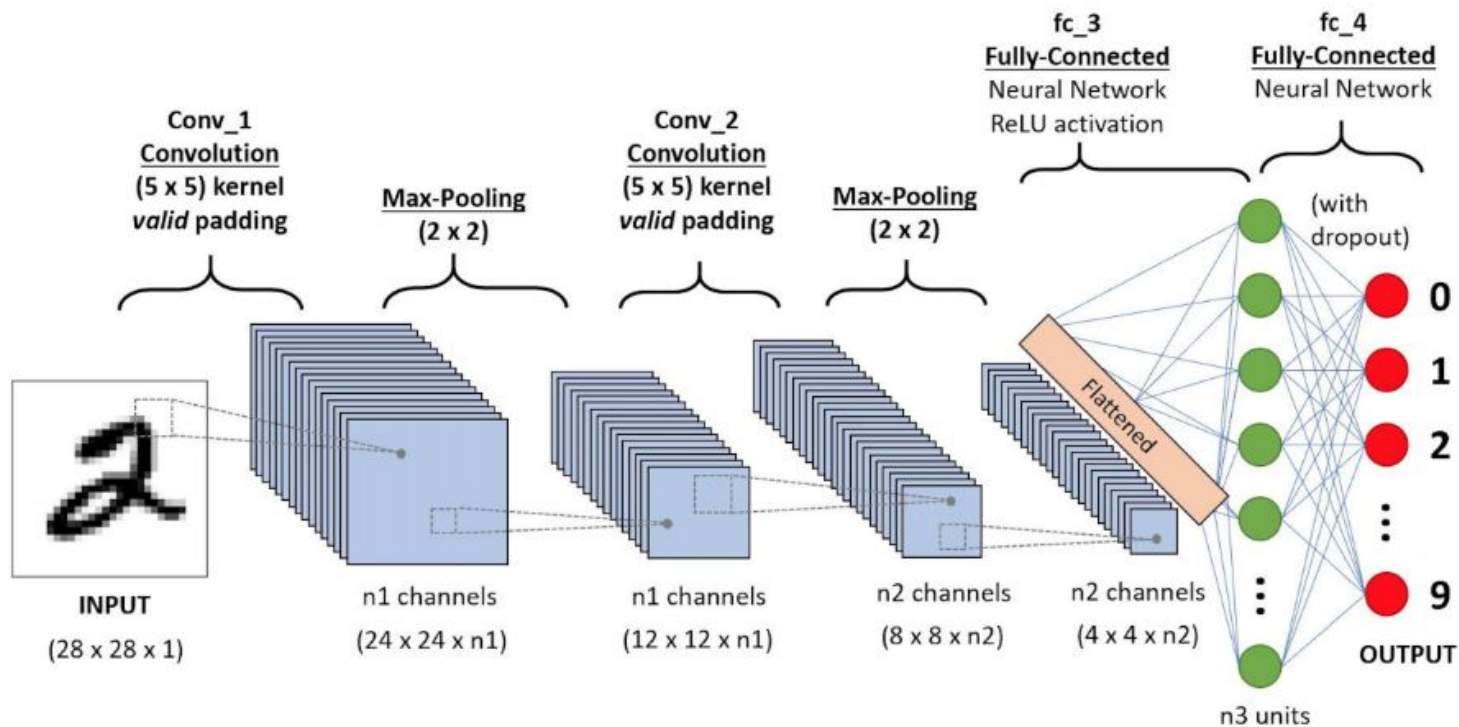
Pooling

- Разбивает изображение на участки $n \times m$ и считает некоторую статистику в каждом участке (обычно максимум)
- Существенно сокращает размер изображения (значит, увеличивает поле восприятия следующих слоёв)
- Не имеет параметров

Зачем это всё?

- Важно следить за тем, чтобы последние свёрточные слои имели размер поля восприятия, сравнимый со всей картинкой

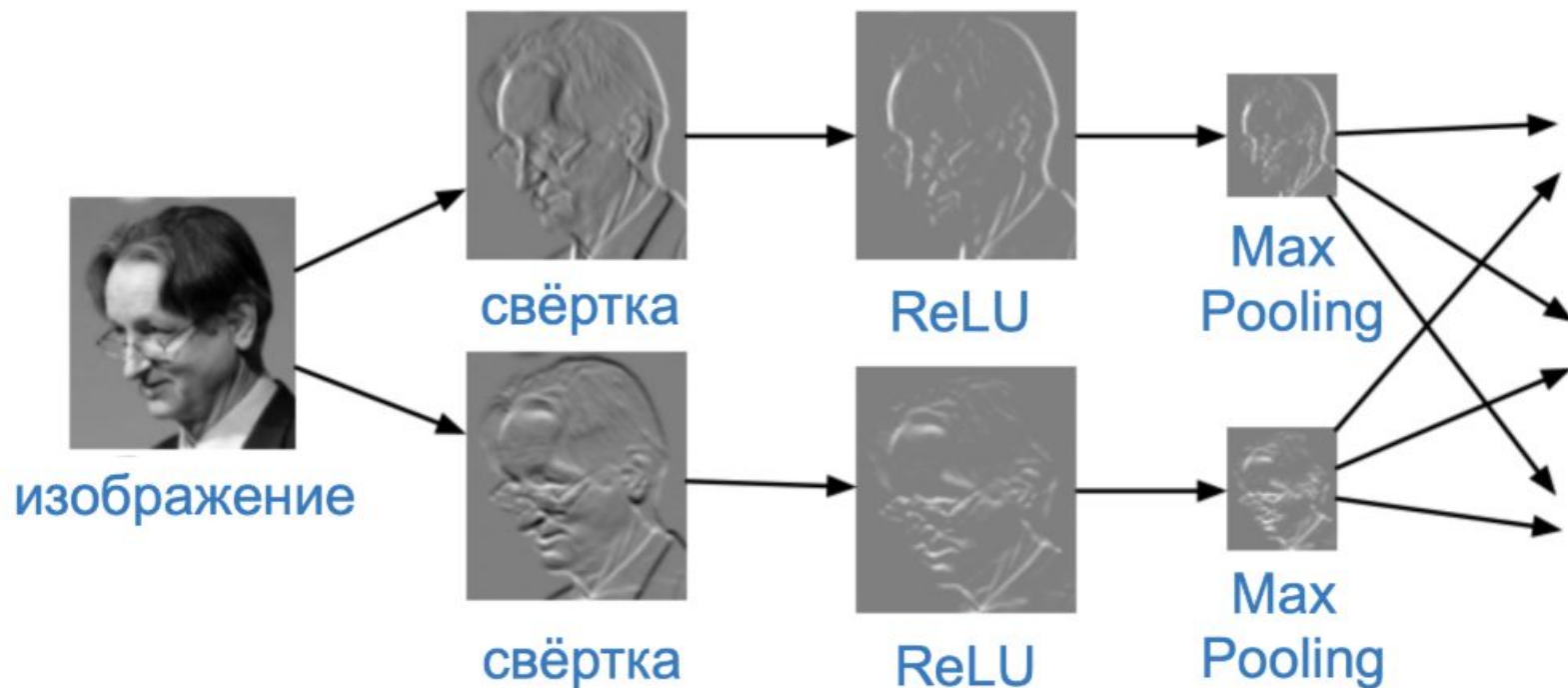
Типичная архитектура



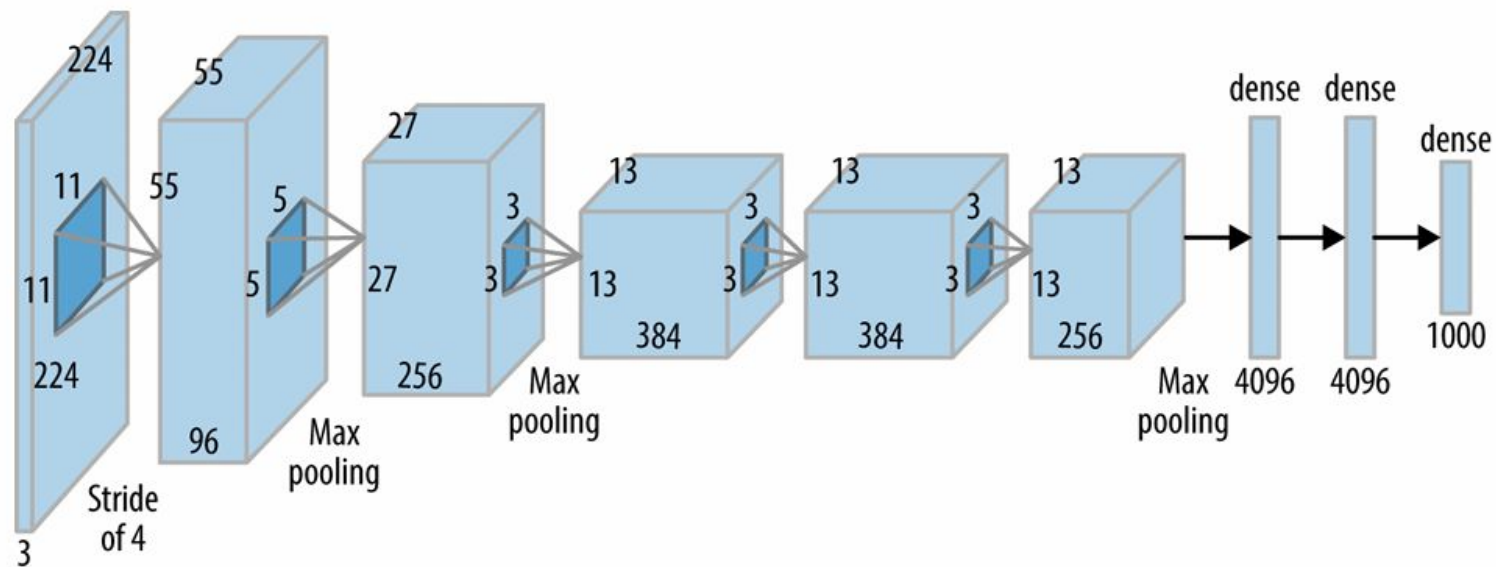
Типичная архитектура

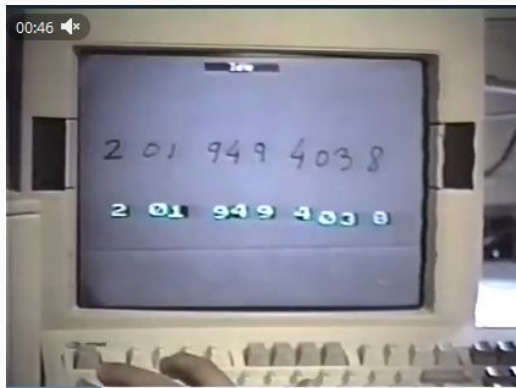
- Последовательное применение комбинаций вида: свёрточный слой $\Rightarrow$ нелинейность $\Rightarrow$ pooling
- Выпрямление (flattening) выхода очередного слоя
- Серия полносвязных слоёв

Типичная архитектура



AlexNet





👉 На видео 32-летний Янн Лекун демонстрирует первую в мире сверточную нейронную сеть (CNN) для распознавания текста в 1993 году.

📅 Когда появились традиционные методы обработки изображений:

Традиционные (или классические) методы начали развиваться с 1960-х годов, а активно применяться — с 1970–1980-х, задолго до появления современных нейросетей.

✅ К таким методам относятся:

- Фильтрация изображений (Гаусс, Собель, Лаплас и др.)
- Детектирование границ (Canny, Prewitt)
- Морфологическая обработка (эрозия, дилатация)
- Бинаризация, сегментация, пороговая фильтрация
- Шумоподавление, выделение контуров

📌 К 1990-м эти техники уже активно использовались в промышленности, медицине, военной технике и OCR (распознавании текста), например в факсах и сканерах. Именно в этом контексте и появлялись первые попытки заменить часть ручной обработки нейросетями, как сделал Лекун с CNN.